

GUÍA PRÁCTICA DE ALPINE.JS

Cómo replicar, paso a paso y con código, el patrón usado
en el proyecto ARTISYNC (login, registro y panel administrativo)

Incluye fragmentos de código listos para copiar y adaptar

Tabla de Contenido

Tabla de Contenido	2
Introducción: cómo usar esta guía	3
Requisitos previos	3
Estructura de archivos que vamos a construir.....	3
Paso 0 — Incluir Alpine.js en el HTML base	3
Paso 1 — Cliente API reutilizable (api-client.js).....	3
Paso 2 — Componente global reutilizable: modo oscuro/claro	4
Paso 3 — Formulario simple: Login	5
3.1 HTML del formulario	5
3.2 JavaScript del componente	6
Receta general para cualquier formulario simple	6
Paso 4 — Formulario avanzado: validaciones con getters computados	6
4.1 HTML: mostrar pistas de validación en vivo	7
4.2 JavaScript: los tres getters	7
Receta: cómo agregar tu propia validación.....	8
Paso 5 — Selección múltiple: checkboxes ligados a un array	8
Paso 6 — Tabla dinámica con datos desde la API	8
Receta: cómo replicar cualquier tabla paginada	9
Paso 7 — Modal de edición controlado por una bandera	10
Paso 8 — Acciones rápidas con confirmación (suspender / eliminar).....	10
Paso 9 — Exportar datos (CSV / PDF) desde el navegador.....	11

Introducción: cómo usar esta guía

Este documento no es solo teoría: está organizado como una receta paso a paso para levantar, desde cero, un mini-sistema de autenticación y administración con Alpine.js, replicando exactamente el patrón usado en ARTISYNC (login.html, registro.html, crud-usuarios.html y admin-usuarios.html).

Cada paso incluye código real (o casi idéntico al del proyecto) que puedes copiar y adaptar cambiando nombres de campos, endpoints y estilos. El orden de los pasos es intencional: primero la base (cliente API y tema), luego un formulario simple, luego uno con validaciones avanzadas, y por último un panel con tabla, modal y acciones CRUD.

Requisitos previos

- Un editor de texto y un navegador — no se necesita Node.js ni ningún bundler.
- Una API backend que responda JSON (en los ejemplos: <http://localhost:8080/api>).
- Conocimientos básicos de HTML, CSS y JavaScript (funciones, objetos, async/await).

Estructura de archivos que vamos a construir

```
mi-proyecto/
├── api-client.js      ← cliente HTTP + manejo de sesión (Paso 1)
├── login.html        ← formulario simple (Paso 3)
├── registro.html     ← formulario con validaciones avanzadas (Paso 4)
└── panel-admin.html  ← tabla + modal + acciones CRUD (Pasos 6-8)
```

Paso 0 — Incluir Alpine.js en el HTML base

Todas las páginas siguen el mismo esqueleto: Alpine se carga por CDN con el atributo defer (obligatorio), y justo después se incluye el cliente API que construiremos en el Paso 1.

```
<!DOCTYPE html>
<html lang="es" data-theme="oscuro">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Mi Proyecto</title>

  <!-- Alpine.js: SIEMPRE con "defer" -->
  <script defer src="https://cdn.jsdelivr.net/npm/alpinejs@3.x.x/dist/cdn.min.js"></script>

  <!-- Nuestro cliente API (Paso 1), se carga ANTES de que Alpine lo necesite -->
  <script src="api-client.js"></script>
</head>
<body x-data="temaGlobal()">
  <!-- aquí va el contenido -->
</body>
</html>
```

Tip para evitar parpadeos: agrega esta regla CSS una sola vez en cada página. Oculta cualquier elemento con x-cloak hasta que Alpine termine de inicializar.

```
<style>[x-cloak] { display: none !important; }</style>
```

Paso 1 — Cliente API reutilizable (api-client.js)

Este archivo NO usa Alpine: es JavaScript plano que se reutiliza desde cualquier componente. Resuelve dos problemas para todo el proyecto: dónde guardar el token de sesión, y cómo inyectarlo automáticamente en cada petición.

```
// api-client.js
const API_BASE_URL = 'http://localhost:8080/api';

const authStore = {
  getToken() {
    return localStorage.getItem('accessToken');
  },
  setToken(token) {
    localStorage.setItem('accessToken', token);
  },
  removeToken() {
    localStorage.removeItem('accessToken');
  }
};

async function fetchWithAuth(url, options = {}) {
  const token = authStore.getToken();

  const headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    ...options.headers,
  };

  if (token) {
    headers['Authorization'] = `Bearer ${token}`;
  }

  let response = await fetch(`${API_BASE_URL}${url}`, { ...options, headers });

  if (response.status === 401) {
    authStore.removeToken();
    if (!window.location.pathname.includes('login.html')) {
      window.location.href = 'login.html';
    }
  }

  return response;
}
```

Cómo replicarlo: copia este archivo tal cual, cambia solo API_BASE_URL por la URL de tu backend. Cualquier componente Alpine podrá llamar fetchWithAuth('/lo-que-sea', { method: 'POST', body: ... }) sin preocuparse por el token.

Paso 2 — Componente global reutilizable: modo oscuro/claro

Este es el primer componente Alpine del proyecto. Se declara en el <body> de TODAS las páginas para que el tema se mantenga consistente en toda la aplicación.

```
<body x-data="temaGlobal()">
  <button class="theme-toggle"
    @click="toggleTema()"
    x-text="tema === 'oscuro' ? '☀ Claro' : '🌙 Oscuro'">
  </button>

  <!-- resto del contenido -->
```

```

<script>
  function temaGlobal() {
    return {
      tema: localStorage.getItem('tema') || 'oscuro',

      // init() se ejecuta automáticamente al montar el componente
      init() {
        document.documentElement.setAttribute('data-theme', this.tema);
      },

      toggleTema() {
        this.tema = this.tema === 'oscuro' ? 'claro' : 'oscuro';
        document.documentElement.setAttribute('data-theme', this.tema);
        localStorage.setItem('tema', this.tema);
      }
    }
  }
</script>
</body>

```

Cómo replicarlo: pega este bloque de <script> en cada página nueva y usa las variables --bg, --card, --text, etc. definidas en :root y en [data-theme="oscuro"] dentro de tu CSS, tal como hace ARTISYNC.

Paso 3 — Formulario simple: Login

Con la base lista, el primer formulario real. Este patrón (form + cargando + error + success) se repite en todo el proyecto y es el bloque más reutilizable de toda la guía.

3.1 HTML del formulario

```

<div class="container" x-data="loginForm()">
  <div class="card">

    <!-- Alerta de error: oculta hasta inicializar (x-cloak),
         se muestra/oculta con x-show, y anima con x-transition -->
    <div x-show="error" class="alert alert-error" x-cloak x-transition>
      <span x-text="error"></span>
    </div>

    <div x-show="success" class="alert alert-success" x-cloak x-transition>
      <span>¡Sesión iniciada! Redirigiendo...</span>
    </div>

    <form @submit.prevent="submitForm" x-show="!success">
      <div class="form-group">
        <label for="correo">Correo Electrónico</label>
        <input type="email" id="correo" x-model="form.correo" required>
      </div>

      <div class="form-group">
        <label for="contrasena">Contraseña</label>
        <input type="password" id="contrasena" x-model="form.contrasena" required>
      </div>

      <button type="submit" class="btn-primary" :disabled="cargando">
        <span x-show="!cargando">Iniciar Sesión</span>
        <span x-show="cargando">Comprobando...</span>
      </button>
    </form>
  </div>
</div>

```

3.2 JavaScript del componente

```
function loginForm() {
  return {
    form: { correo: '', contraseña: '' },
    cargando: false,
    error: null,
    success: false,

    async submitForm() {
      this.error = null;
      this.cargando = true;

      try {
        const response = await fetch(`${API_BASE_URL}/auth/login`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify(this.form)
        });

        const data = await response.json();

        if (!response.ok) {
          throw new Error(data.message || 'Credenciales incorrectas');
        }

        if (data.accessToken) {
          authStore.setToken(data.accessToken);
        }

        this.success = true;
        setTimeout(() => { window.location.href = 'panel-admin.html'; }, 1000);

      } catch (err) {
        this.error = err.message;
      } finally {
        this.cargando = false;
      }
    }
  }
}
```

Receta general para cualquier formulario simple

- 1. Declara form: {} con un campo por cada input, más cargando, error y success.
- 2. Enlaza cada input con x-model="form.campo".
- 3. Intercepta el envío con @submit.prevent="submitForm" en el <form>.
- 4. Dentro de submitForm(): pon cargando = true, haz el fetch dentro de try/catch/finally, y actualiza error o success según el resultado.
- 5. En el botón, usa :disabled="cargando" y dos con x-show="!cargando" / x-show="cargando" para cambiar el texto sin tocar el DOM manualmente.

Paso 4 — Formulario avanzado: validaciones con getters computados

Cuando el formulario necesita reglas de negocio (edad mínima, fuerza de contraseña, habilitar/deshabilitar el botón), la forma correcta en Alpine es usar getters de JavaScript dentro del objeto x-data: Alpine los vuelve a calcular automáticamente cada vez que cambian las variables de las que dependen.

4.1 HTML: mostrar pistas de validación en vivo

```
<div class="form-group">
  <label for="contrasena">Contraseña</label>
  <input type="password" id="contrasena" x-model="form.contrasena" required minlength="8">

  <span class="validation-hint"
    x-show="form.contrasena.length > 0 && !contrasenaFuerte"
    x-cloak>
    Debe contener al menos 8 caracteres, un número, una mayúscula y una minúscula.
  </span>
</div>

<div class="form-group">
  <label for="fechaNacimiento">Fecha de Nacimiento</label>
  <input type="date" id="fechaNacimiento" x-model="form.fechaNacimiento" required>

  <span class="validation-hint" x-show="form.fechaNacimiento && !esMayorDeEdad" x-cloak>
    Debes tener al menos 18 años para registrarte.
  </span>
</div>

<button type="submit" class="btn-primary" :disabled="!esValido || cargando">
  <span x-show="!cargando">Crear Cuenta</span>
  <span x-show="cargando">Procesando...</span>
</button>
```

4.2 JavaScript: los tres getters

```
function registroForm() {
  return {
    form: {
      nombres: '', apellidos: '', correo: '', contrasena: '',
      fechaNacimiento: '', rol: 'CLIENTE', aceptaTerminos: false
    },
    cargando: false, error: null, success: false,

    // 1) Getter: ¿cumple 18 años?
    get esMayorDeEdad() {
      if (!this.form.fechaNacimiento) return false;
      const hoy = new Date();
      const fechaNac = new Date(this.form.fechaNacimiento);
      let edad = hoy.getFullYear() - fechaNac.getFullYear();
      const mes = hoy.getMonth() - fechaNac.getMonth();
      if (mes < 0 || (mes === 0 && hoy.getDate() < fechaNac.getDate())) edad--;
      return edad >= 18;
    },

    // 2) Getter: ¿la contraseña es fuerte?
    get contrasenaFuerte() {
      const regex = /^(?=.*[0-9])(?=.*[a-z])(?=.*[A-Z]).{8,}$/;
      return regex.test(this.form.contrasena);
    },

    // 3) Getter final: combina los anteriores + campos obligatorios
    get esValido() {
      return this.form.nombres.length > 0 &&
        this.form.apellidos.length > 0 &&
        this.form.correo.includes('@') &&
        this.contrasenaFuerte &&
        this.esMayorDeEdad &&
        this.form.aceptaTerminos;
    },
  },
}
```

```

    async submitForm() {
      // mismo patrón try/catch/finally del Paso 3
    }
  }
}

```

Receta: cómo agregar tu propia validación

- Escribe un `get miValidacion() { ... }` que retorne `true/false` leyendo `this.form.*`.
- Úsalo directamente en el HTML: `x-show="!miValidacion"` para pistas, o dentro de otro getter (`esValido`) para combinarlo con otras reglas.
- No necesitas ningún watcher manual: al ser un getter, Alpine lo reevalúa solo cuando cambia una variable involucrada.

Paso 5 — Selección múltiple: checkboxes ligados a un array

Cuando necesitas que el usuario marque varias opciones (por ejemplo, roles del sistema), basta con inicializar un array vacío y enlazar cada checkbox con el mismo `x-model`.

```

<div class="checkbox-grid">
  <template x-for="rol in rolesDisponibles" :key="rol">
    <label class="check-item">
      <input type="checkbox" :value="rol" x-model="form.roles">
      <span x-text="rol"></span>
    </label>
  </template>
</div>

<script>
function adminForm() {
  return {
    form: {
      nombres: '', apellidos: '', correo: '', contrasena: '',
      roles: [] // Alpine hace push/splice automático aquí
    },
    rolesDisponibles: ['ADMIN', 'CLIENTE', 'CREADOR', 'SOPORTE', 'AUDITORIA_EXTERNA'],
    // ...
  }
}
</script>

```

Cómo funciona: al marcar un checkbox, Alpine agrega automáticamente el valor de `:value` al array `form.roles`; al desmarcarlo, lo quita. No hace falta escribir ninguna lógica de `push()` o `splice()` manual.

Paso 6 — Tabla dinámica con datos desde la API

El panel administrativo necesita listar registros que vienen paginados desde el backend. El patrón: `init()` carga la primera página, `x-for` pinta las filas, y las variables `page/size/totalPages` controlan la paginación.

```

<div class="main-content" x-data="crudUsuarios()">
  <table>
    <thead>
      <tr><th>Nombre</th><th>Correo</th><th>Estado</th><th>Acciones</th></tr>
    </thead>
    <tbody>
      <template x-for="user in usuarios" :key="user.idUsuario">
        <tr>
          <td x-text="user.nombres + ' ' + user.apellidos"></td>
          <td x-text="user.correo"></td>
          <td>

```



```

        <span class="badge" :class="'badge-' + user.estadoCuenta"
            x-text="user.estadoCuenta"></span>
    </td>
    <td class="table-actions">
        <button class="btn btn-outline" @click="abrirEdicion(user)">Editar</button>
        <button class="btn btn-danger"
@click="eliminarUsuario(user.idUsuario)">Eliminar</button>
    </td>
</tr>
</template>
</tbody>
</table>

<div class="pagination">
    <button @click="page--; cargarUsuarios()" :disabled="page === 0">Anterior</button>
    <span x-text="'Página ${page + 1} de ${totalPages}'"></span>
    <button @click="page++; cargarUsuarios()" :disabled="page + 1 >=
totalPages">Siguiente</button>
</div>
</div>

<script>
function crudUsuarios() {
    return {
        usuarios: [],
        page: 0,
        size: 10,
        totalPages: 1,
        cargando: false,

        // init() es llamado automáticamente por Alpine al montar
        init() {
            this.cargarUsuarios();
        },

        async cargarUsuarios() {
            this.cargando = true;
            try {
                const res = await fetchWithAuth(
`/admin/usuarios?page=${this.page}&size=${this.size}&sortBy=idUsuario&direction=desc`
                );
                if (res.ok) {
                    const data = await res.json();
                    this.usuarios = data.content;
                    this.totalPages = data.totalPages || 1;
                }
            } finally {
                this.cargando = false;
            }
        }
    }
}
</script>

```

Receta: cómo replicar cualquier tabla paginada

- Define usuarios: [] (o el nombre de tu recurso), page, size y totalPages.
- En init(), llama a tu función de carga para que la tabla no aparezca vacía al entrar.
- Pinta las filas con <template x-for="item in items" :key="item.id">, siempre con :key único.
- Los botones de paginación solo cambian page y vuelven a llamar a la función de carga.

Paso 7 — Modal de edición controlado por una bandera

El patrón más simple y más usado en Alpine para modales: una variable booleana (`modalAbierto`) decide si se muestra, y un objeto temporal (`formEdit`) guarda los datos que se están editando sin tocar todavía el array original.

```
<!-- Botón que abre el modal -->
<button class="btn btn-outline" @click="abrirEdicion(user)">Editar</button>

<!-- Modal -->
<div class="modal-overlay" x-show="modalAbierto" x-cloak x-transition>
  <div class="card modal-content" @click.outside="modalAbierto = false">
    <h3 class="modal-title">Editar usuario</h3>

    <div class="form-group">
      <label class="form-label">Nombres</label>
      <input class="form-input" x-model="formEdit.nombres">
    </div>
    <div class="form-group">
      <label class="form-label">Apellidos</label>
      <input class="form-input" x-model="formEdit.apellidos">
    </div>

    <div class="modal-actions">
      <button class="btn btn-outline btn-flex" @click="modalAbierto = false">Cancelar</button>
      <button class="btn btn-primary btn-flex" @click="guardarEdicion()">Guardar</button>
    </div>
  </div>
</div>

<script>
// Dentro de crudUsuarios():
modalAbierto: false,
formEdit: { id: null, nombres: '', apellidos: '' },

abrirEdicion(user) {
  this.formEdit.id = user.idUsuario;
  this.formEdit.nombres = user.nombres;
  this.formEdit.apellidos = user.apellidos;
  this.modalAbierto = true;
},

async guardarEdicion() {
  const res = await fetchWithAuth(`/admin/usuarios/${this.formEdit.id}`, {
    method: 'PUT',
    body: JSON.stringify(this.formEdit)
  });
  if (res.ok) {
    this.modalAbierto = false;
    this.cargarUsuarios(); // refresca la tabla
  }
}
</script>
```

@click.outside="modalAbierto = false": modificador de Alpine que cierra el modal automáticamente si el usuario hace clic fuera de la tarjeta.

Paso 8 — Acciones rápidas con confirmación (suspender / eliminar)

Para acciones destructivas o sensibles, el proyecto usa `confirm()` nativo antes de llamar a la API. Es la forma más simple de añadir una confirmación sin construir un modal adicional.

```
<button class="btn btn-warning"
  @click="cambiarEstado(user.idUsuario, 'SUSPENDIDO')">
  Suspende
</button>

<script>
async cambiarEstado(id, nuevoEstado) {
  if (!confirm(`¿Cambiar el estado a ${nuevoEstado}?`)) return;

  const res = await fetchWithAuth(`/admin/usuarios/${id}/estado`, {
    method: 'PATCH',
    body: JSON.stringify({ nuevoEstado })
  });

  if (res.ok) {
    this.cargarUsuarios();
  } else {
    alert('Error cambiando estado: ' + (await res.text()));
  }
}
</script>
```

Mejora recomendada: si quieres una experiencia más cuidada (sin el estilo del navegador), reemplaza `confirm()/alert()` por el mismo patrón de modal booleano del Paso 7, mostrando un mensaje de "¿Estás seguro?" con dos botones.

Paso 9 — Exportar datos (CSV / PDF) desde el navegador

Estas funciones no usan directivas especiales de Alpine: son métodos normales del componente que se invocan igual que cualquier otro con `@click`. La diferencia es que arman el archivo en el propio navegador.

```
<button class="btn btn-outline" @click="exportarCSV()">Exportar CSV</button>

<script>
async exportarCSV() {
  const res = await fetchWithAuth('/admin/usuarios?page=0&size=1000');
  const data = await res.json();

  const filas = data.content.map(u => `${u.nombres},${u.apellidos},${u.correo}`);
  const csv = 'Nombres,Apellidos,Correo\n' + filas.join('\n');

  const blob = new Blob([csv], { type: 'text/csv' });
  const url = URL.createObjectURL(blob);
  const a = document.createElement('a');
  a.href = url;
  a.download = 'usuarios.csv';
  a.click();
  URL.revokeObjectURL(url);
}
</script>
```

Para PDF, el proyecto usa las librerías externas `jsPDF` y `jspdf-autotable` (cargadas por `<script>` antes que `api-client.js`), y llama a `doc.autoTable({...})` con las mismas filas obtenidas de la API.